

COM3021 – Data Science at Scale

High-performance computing

DR YANDA MENG



Introduction

- High-performance computational systems;
 - Processing power*
 - Communication*
 - I/O*
 - Supercomputers
 - Can be distributed or non-distributed
 - Processing power is mostly driven by the size of cluster in terms of processor cores and their speeds
 - Number of nodes*
 - Processors per nodes*
 - Cores per processor*
-

Performance

- Processing power in FLOPS (floating point operations per second)

gigaFLOPS *GFLOPS* 10^9

teraFLOPS *TFLOPS* 10^{12}

petaFLOPS *PFLOPS* 10^{15}

exaFLOPS *EFLOPS* 10^{18}

- Smartphones: 1—2 GFLOPS;
 - Laptops and desktops: 10 GFLOPS;
 - High-end servers: ~25—100 GFLOPS;
 - GPUs: 20 GFLOPS — 2 TFLOPS;
-

Performance

- Network performance

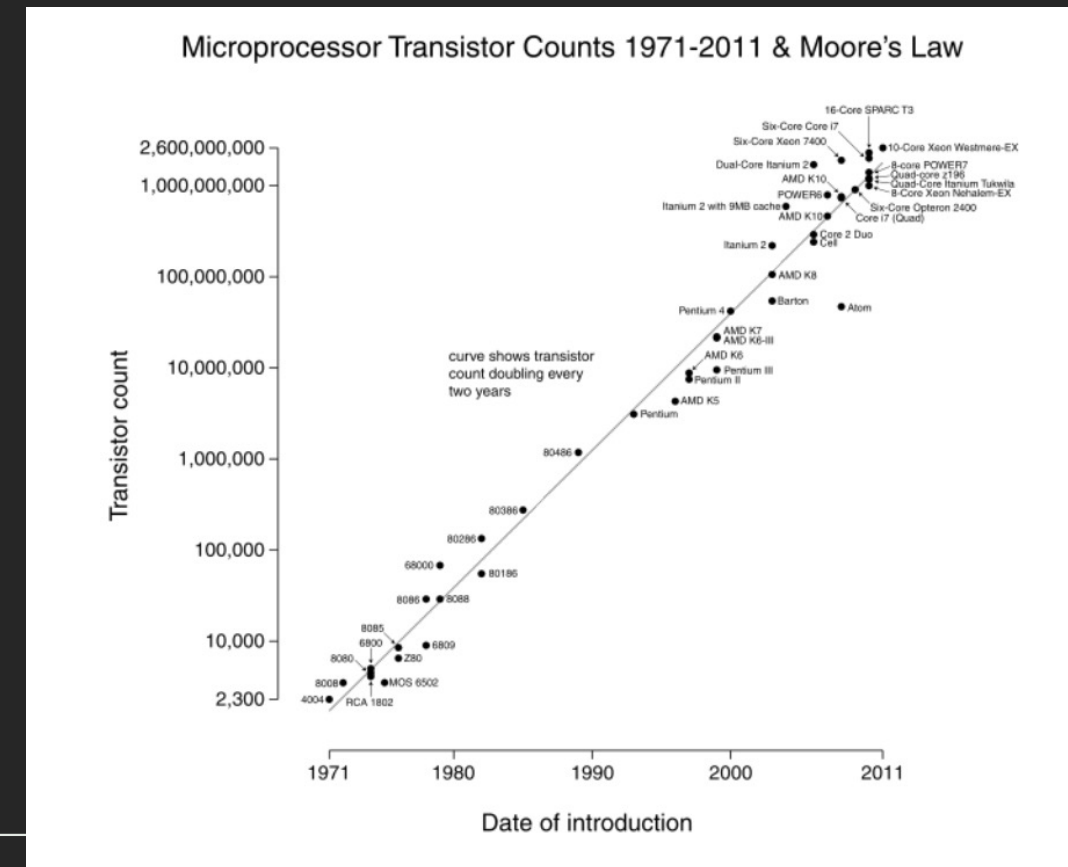
Important piece for distributed systems

LAN (Local-area Network)

Name and Year	Bandwidth
10BASE Ethernet (1990)	10 Mbps
100BASE Ethernet (1995)	100 Mbps
1000BASE Ethernet (1999)	1 Gbps
Myrinet-10G (2005)	10 Gbps
10GBASE Ethernet (2006)	10 Gbit/s
Infiniband 12X QDR (2009)	96 Gbit/s
100GBASE Ethernet (2010)	100 Gbit/s

Moore's Law

- “The observation that over the history of computing hardware, the number of transistors on integrated circuits doubles approximately every two years.”
- The industry is hitting its limits and lagging behind Moore's law



Amdahl's Law

Predicts the theoretical speedup when using multiple processors

- The speedup is bounded by the sequential part of a computation that cannot be parallelised

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

- S_{latency} is the theoretical speedup of the execution of the whole task;
 - s is the speedup of the part of the task that benefits from improved system resources;
 - p is the proportion of execution time that the part benefiting from improved resources originally occupied.
-

Amdahl's Law

Ex: Workload: sum of 10 scalars (sequential), and
10 × 10 matrix sum (parallel)

- Single processor:

$$Time = (10 + 100) \times t^{add}$$

- 10 processors

$$Time = 10 \times t^{add} + 100/10 \times t^{add} = 20 \times t^{add}$$

$$S_{\text{latency}}(s) = \frac{1}{(1 - p) + \frac{p}{s}}$$

- S_{latency} is the theoretical speedup of the execution of the whole task;
 - s is the speedup of the part of the task that benefits from improved system resources;
 - p is the proportion of execution time that the part benefiting from improved resources originally occupied.
-

Amdahl's law

Example: Let a program have 40 percent of its code enhanced (so $f_E = 0.4$) to run 2.3 times faster (so $f_I = 2.3$). What is the overall system speedup S ?

Amdahl's law

Example: Let a program have 40 percent of its code enhanced (so $f_E = 0.4$) to run 2.3 times faster (so $f_I = 2.3$). What is the overall system speedup S ?

Step 1: Setup the equation: $S = ((1 - f_E) + (f_E / f_I))^{-1}$

Step 2: Plug in values & solve $S = ((1 - 0.4) + (0.4 / 2.3))^{-1}$
 $= (0.6 + 0.174)^{-1} = 1 / 0.774$
 $= \mathbf{1.292}$

Resource Management

- The hosting data centre needs to employ dedicated system administrators, pay for support contracts and/or a maintenance crew, and cover the cost of electricity used to power and cool the machine
 - *"cost of ownership"*
 - *Electricity can cost up to \$10M a year*
 - Resource management software plays a critical role in how supercomputing system resources are allocated to user applications.
 - Resource management tools are an inherent part of the high performance computing (HPC) software stack. They perform three main tasks
 - *resource allocation,*
 - *workload scheduling*
 - *support for distributed workload execution and monitoring.*
-

Resource Management

- *Resource allocation* takes care of assigning physical hardware, which may span from a fraction of the machine to the entire system, to specific user tasks based on their requirements. Resource managers typically recognize the following resource types.
 - *Compute nodes*
 - Processing cores
 - Memory
 - Storage
 - *Interconnect*
-

Resource Management

- Resource managers allocate the available computing resources to jobs specified by users.
 - A job is a self-contained work unit with associated input data that during its execution produces some output result data.
 - Jobs may be monolithic or subdivided into a number of smaller steps or tasks.
 - Jobs may be executed interactively or batch processed.
 - Pending jobs are stored in queues.
 - The job queue defines the order in which jobs are selected by the resource manager for execution.
-

HPC

HPC

- Use of supercomputers and high-capacity computing resources to solve complex computational problems.

parallel processing

clustering

specialized hardware (e.g., GPUs)

High levels of performance and throughput.

Why is it relevant for DS?

- Big Data Handling

In the era of big data, handling vast amounts of information efficiently is a challenge.

HPC systems excel in managing and processing enormous datasets, enabling data scientists to work with more data and gain deeper insights.

- Complex Analytics

HPC provides the computational muscle required to run these models at scale, reducing the time it takes to derive meaningful results.

Why is it relevant for DS?

- Machine Learning and Deep Learning

Training machine learning and deep learning models often demands extensive computational resources.

HPC clusters equipped with GPUs can accelerate model training, making experimentation faster and more productive.

- Simulations and Scientific Research

Data science isn't limited to business analytics. It's also applied in scientific research, where simulations of physical phenomena or climate modelling require immense computational power provided by HPC.

Key aspects

- Parallel computing
 - Cluster computing
 - Heterogeneous computing
 - Scalability
 - Data management
-

Tools and libraries

- Parallel programming

OpenMPI, OpenMP

- Deep learning

CUDA, cuDNN

- Parallel file systems

Lustre, GPFS

- Linear algebra libraries

BLAS, LAPACK

Tools and libraries

- General purpose languages and libraries

Python, Numpy, Cython

- Job Schedulers

SLURM, Torque

- Performance profiling and debugging

GDB, Valgrind

- Visualization and Data Analysis

ParaView, VisIt, Matplotlib

SLURM

- *Slurm Workload Manager (formerly Simple Linux Utility for Resource Management (SLURM))* is an open-source, modular, extensible, scalable resource manager and workload scheduling software for clusters and supercomputers
-

SLURM

Workload organisation

Nodes are divided into logical sets called partitions.

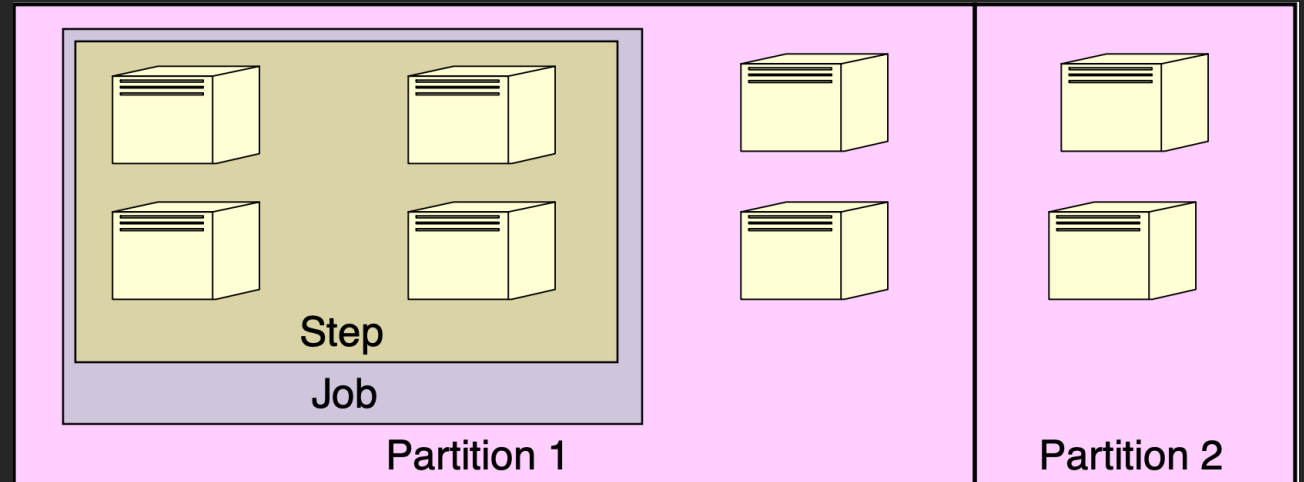
Partitions in SLURM represent individual job queues and thus impose specific constraints on user jobs.

The scheduler assigns the available nodes to the highest-priority jobs until the pool of available nodes is exhausted.

The individual tasks composing a job, called job steps, may utilize the entire set of nodes allocated to the parent job or only a fraction.

SLURM entities

- Nodes: Individual computers
- Partitions: Job queues
- Jobs: Resource allocations
- Job steps: Set of (typically parallel) tasks



Nodes

- Configuration parameters

Processor count

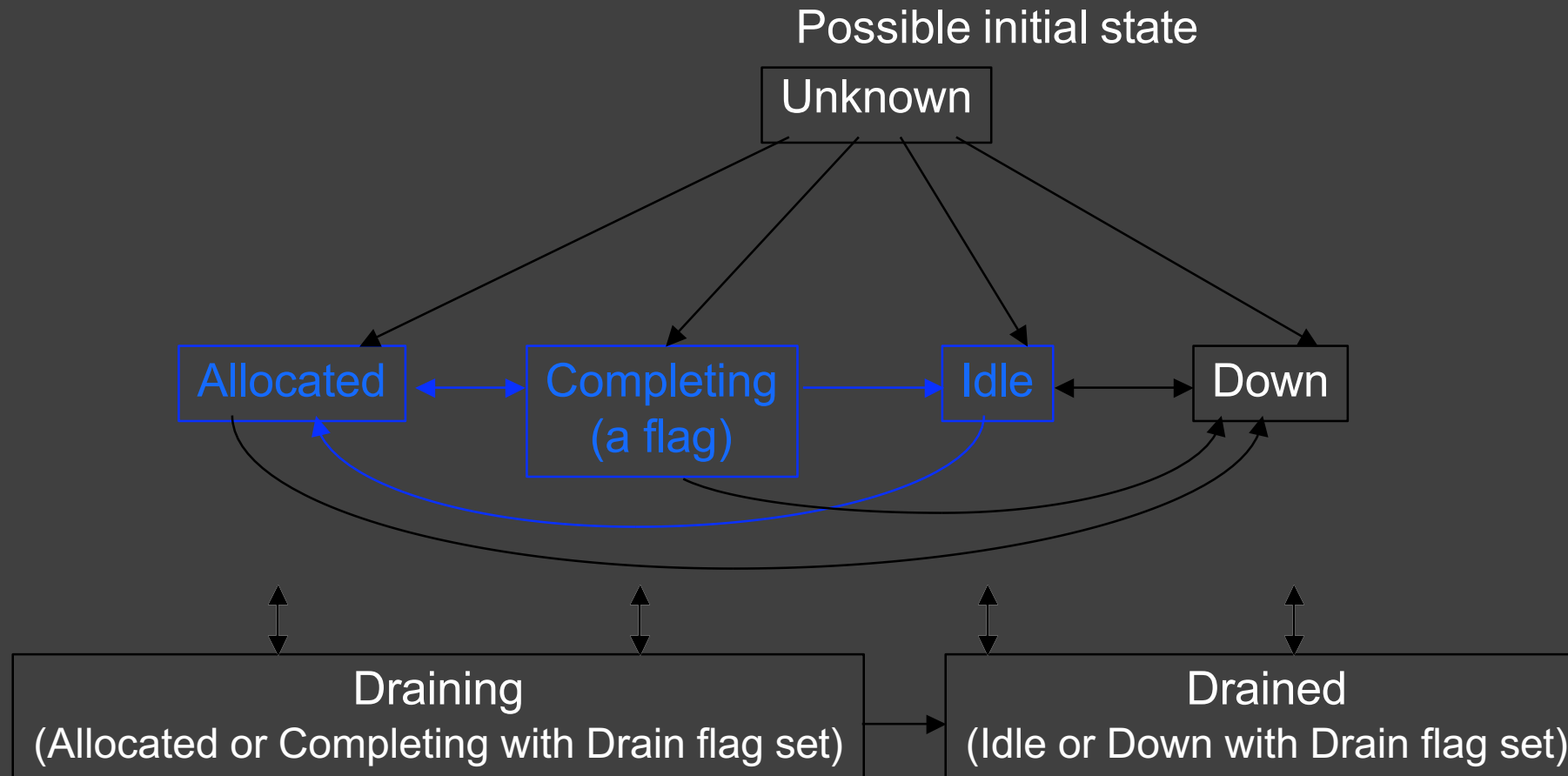
Real memory size

Temporary disk space

Features (arbitrary string, e.g. OS version)

Scheduling weight (preference to allocate, use smaller number for less capable nodes to use them first unless users specifically request more capable nodes, e.g. more memory)

Node states



scontrol update NodeName=X state=[drain | resume] Reason=X

Jobs

- Resource allocation: specific processors and memory or entire nodes allocated to a user for some time period
 - Can be interactive (executed in real-time) or batch (script queued for later execution)
 - Many constraints available for user request
 - Identified by ID number
-

Job states

